TRADING ENGINE

Project Plan v2

L3 replay, exact C++ data contracts, and defensible corpus validation

Fuaad Bashi

Revised 9 August 2026

Primary outcome: reconstruct a gap-aware Bitstamp L3 book and evaluate fill-within-horizon forecasts on held-out observed orders, with calibration, censoring, uncertainty and limitations stated explicitly.

CURRENT POSITION

Slice 0 complete. Segmented capture and validation complete. Strong C++ value types complete with 11 passing tests. Normalized event design is the current red TDD step.

Contents

Owner: Fuaad Bashi **Revised:** 2026-08-09 **Purpose:** Learn modern C++ and low-latency systems by building a defensible L3 replay, book-reconstruction and execution-model validation system, then expose the same historical and live engine through an interactive local web dashboard.

This replaces the venue, validation and schedule assumptions in the August 2026 plan. The original plan remains useful for its C++ curriculum and slice discipline; this document is the current source of truth when the two disagree.

Section 1 - The revised deliverable

The GUI is part of the final deliverable, but it is not evidence that the engine is correct. The primary technical result is a reproducible evaluation built from held-out Bitstamp L3 sessions:

> *Reconstruct a gap-aware order book from raw venue data, make out-of-sample forecasts about whether observed resting orders fill within fixed horizons, and report calibration, time-to-fill and adverse-selection results with confidence intervals and explicit limitations.*

The strongest final README claim should have this form:

> *Across N eligible orders in M held-out capture sessions, the model achieved Brier score X for fill-within-Delta, with calibration error Y. Median time-to-fill error was Z, and post-fill adverse selection was concentrated in named spread/volatility regimes. Sessions containing a gap were censored or excluded according to the documented policy.*

Numbers are placeholders until measured. The project must never promise that a historical replay proves how a hypothetical order would have changed the market.

The final showcase adds a local browser-based dashboard over that validated engine. It must let a user replay old sessions, observe new live market data, submit safe local paper orders and compare model predictions with later outcomes. Historical and live modes must use the same C++ book, queue model and execution interfaces; Python displays and controls them but does not reimplement their financial logic.

Section 2 - What is and is not ground truth

Primary labels: outcomes of real observed orders

For a real `order_created` event at time T, use only state available at T to predict whether that specific order will fill within one or more horizons. Label its later outcome using Bitstamp order events joined to Bitstamp trades by order ID. This avoids pretending an invented order actually existed.

Required labels include:

- filled within 1 s, 5 s and 30 s;
- time to first fill and, where observable, complete fill;
- cancellation or censoring before the horizon;
- signed mid-price movement after a fill (adverse selection).

Secondary experiment: hypothetical queue insertion

A simulated order may be inserted at the back of a price-level queue at T and replayed forward. This is useful sensitivity analysis, but its conclusion is conditional on:

- no market impact from the hypothetical order;
- visible displayed liquidity being complete;
- correct venue price-time priority semantics;
- no hidden/iceberg behavior that changes priority;
- an unbroken, correctly aligned capture segment.

Report this as a counterfactual replay result, not literal fill ground truth.

Live or sandbox execution

External order submission is a secondary integration test. It validates authentication, state transitions, idempotency, risk checks and telemetry. Sandbox fills do not validate production queue dynamics because a sandbox book is artificial. Bitstamp test-server/sandbox access must be proven with a small probe before it enters the critical path; documentation showing test REST URLs is not proof that the required account, WebSocket and execution workflow is available.

Section 3 - Venue and data architecture

Primary venue: Bitstamp

Use the following same-venue data together:

- `live_orders_btcusd`: individual order create/change/delete lifecycle;
- `live_trades_btcusd`: trades with maker/taker order identifiers for fill classification;
- `group=2` REST snapshot: initial L3 state for every continuous segment;
- Bitstamp L2/order-book stream or checkpoints: exact aggregated reconstruction comparison.

The current capture contains `live_orders` plus a `group=2` snapshot. Adding `live_trades` is a required data-contract upgrade before fill validation. Capturing trades and orders requires an explicit synchronization/segmentation policy; timestamps and order IDs must be joined without assuming arrival order across channels.

Secondary venue: Coinbase

Coinbase `level2_batch` remains useful for a second adapter and broad cross-venue sanity checks. It cannot verify the exact Bitstamp top of book: Coinbase and Bitstamp have different orders, participants, liquidity and prices. Any comparison must be labelled cross-venue, time-aligned and tolerance-based.

Capture validity

Every run is a directory containing an atomic manifest and snapshot-backed segments. For each Bitstamp order event within a segment:

```
current.pre_event_id == previous.event_id
```

A mismatch closes and invalidates the segment for book replay. Recovery starts a new connection and snapshot-backed segment. Raw JSONL contains venue payloads only; manifests contain boundary, gap and provenance metadata.

Section 4 - Evidence already collected

Completed foundation

- CMake C++20 project with `te_core`, thin applications and GoogleTest/CTest.
- ASan and UBSan laboratory; Apple-Clang ASan issue isolated; Homebrew Clang path verified.
- Ubuntu CI, dependency pinning and Slice 0 ADRs.
- Git history and Slice 0 learning-notes PDF.

Captures

- Legacy one-hour Bitstamp capture: 252,374 order events, about 121 MiB, one real event-chain

break at line 162,871 and a known startup alignment defect. It is a valuable negative/fault corpus, not one continuous replayable book.

- Clean segmented capture: `data/raw/bitstamp-btcusd-20260809T100421Z`, 1,433 order events,

29.2 seconds, one snapshot-backed segment, valid manifest and no detected chain break.

- Coinbase unauthenticated L2 sample and the recorded authentication failure for `full`.

C++ Slice 1 progress

- `Price`, `Qty`, `OrderId` and `Side` strong value types exist.
- Equality and price ordering are tested; `test_types` has 11 passing tests.
- Normalized `OrderEvent`/`EventKind` design is the current red TDD step and is not complete.

Section 5 - Core scope and cuts

Must ship

1. Gap-aware, provenance-preserving market-data capture.
2. Exact decimal parsing and per-instrument scale metadata.
3. Venue decoder and versioned explicit binary encoding.
4. L3 book reconstruction with invariants and same-venue L2 checks.
5. Deterministic replay using injected feed, clock and execution venue.
6. Queue/fill model with declared assumptions.
7. Held-out corpus validation with calibration and uncertainty.
8. CI, sanitizers, benchmarks, ADRs and learning notes.
9. Read-only live Bitstamp feed and local paper-order execution.
10. A versioned telemetry/control boundary that does not couple the engine to a GUI toolkit.
11. A local web dashboard for historical replay, live observation and paper-order experiments.

Should ship

- SPSC queue and measured cache-line/ordering experiment.
- Multiple capture sessions across regimes.
- Saved research reports and plots that can be reproduced without the dashboard.
- A second tested instrument after the first instrument is correct.

Stretch only

- External Bitstamp sandbox/test execution adapter.
- ZeroMQ as the specific telemetry transport.
- ML signal, equities adapter or second L3 venue.

The dashboard is required, but it is sequenced after engine validation. The telemetry boundary is also required; ZeroMQ is only one possible implementation. The project may instead choose pybind11, a local WebSocket, pipes or another measured solution without changing engine logic.

Section 6 - Revised slice map

The estimates are planning ranges, not deadlines. A slice closes only when its gate passes.

Slice	Estimate	Ships	Gate
0 Foundations	complete	build, tests, CI, sanitizers	tagged and documented
1 Data contract + recorder	2-3 weeks	validated raw capture, types, decoder, binary corpus	golden tests + clean segment
2 L3 book	3 weeks	deterministic book reconstruction	invariants + same-venue L2 match
3 Replay core	2-3 weeks	single-threaded deterministic end-to-end replay	zero-trade conservation tests
4 Queue + execution model	3 weeks	fill/time model and sensitivity variants	synthetic and historical-order tests
5 Corpus validation	2-3 weeks	held-out metrics, plots, bootstrap/session CIs	no leakage + documented exclusions
6 Operational path	2 weeks	live feed + local paper venue + telemetry contract	measured behavior + sanitizers
7 Interactive dashboard	2-3 weeks	historical replay + live observation + paper-order UI	same engine in both modes
Buffer	2 weeks	debugging, documentation and interview rehearsal	no new features
Stretch	after core	sandbox execution, ZeroMQ transport, ML/equities	never blocks the required dashboard

Section 7 - Slice 1 - data contract and recorder

1A. Capture and provenance - complete for order events

- segmented run directory and atomic manifest;
- drain-before-snapshot ordering;
- event-chain validator;
- clean and intentionally faulty corpora.

Before final validation work, extend the capture contract to include Bitstamp trades and a same-venue L2 comparison source. Do not interrupt current C++ learning to build a large capture platform; first specify the joined schemas and obtain a short validated sample.

1B. Value types and normalized events - current

Required concepts:

- strong types rather than integer aliases;
- scoped enums with explicit underlying types;
- equality/ordering operators and C++20 `<=>`;
- aggregate initialization;
- trivial copyability as an in-memory queue contract, not a serialization guarantee;
- venue timestamp and local receive timestamp units stated in names/types;
- instrument identity present before a second instrument is introduced.

Structural invariants that every build relies on belong as header `static_asserts` with useful messages. Runtime behavior belongs in GoogleTest.

1C. InstrumentSpec and exact decimal parser

This is a separate unit, not hidden inside `types.hpp` or simdjson code.

Requirements:

- per-instrument price and quantity decimal scales;
- direct decimal-string to integer conversion without `double`;
- reject malformed sign/decimal syntax, excess precision and overflow;
- checked wide intermediate for price-times-quantity;
- table-driven tests including BTC/USD (2 price decimals, 8 quantity decimals) and at least one

different synthetic instrument scale.

1D. Offline venue decoder

- simdjson On Demand over committed real fixtures;
- subscription/control messages handled explicitly;
- `price_str`, `amount_str` and `id_str` parsed exactly;
- Bitstamp side/event mapping tested;
- event-chain state belongs outside the pure message-to-event conversion where possible;
- malformed/unknown schema returns a reasoned error, not an exception on the hot path.

1E. Binary format

Write a format specification before a C++ layout. Do not `memcpy(OrderEvent)` to disk. Trivial copyability does not fix padding or endianness.

The recommended container mirrors raw segmentation:

- a versioned file header per binary segment;
- explicit field encoding in a declared byte order;
- a separately encoded snapshot or snapshot reference;
- typed order/trade records;
- manifest-owned gap/reconnect/end reasons;
- checksum/hash and event counts in provenance.

If control records are embedded in one binary stream instead, amend ADR 0006 and define their exact semantics first. Do not maintain two competing boundary models accidentally.

1F. Golden evidence

The stable check is:

```
real JSON fixture -> decoded events -> binary bytes -> decoded events
```

Assert semantic equality of the events and stable expected binary bytes. Do not require the normalized event to regenerate byte-identical source JSON; JSON member order and formatting are not part of the normalized event.

Slice 1 exit gate

- types/events/parser/decoder/record tests pass;
- deliberate red phase observed for rule-bearing tests;
- clean capture produces the expected event count;
- legacy gap fixture is rejected at the exact boundary;
- binary bytes are stable and explicitly endian/versioned;
- ASan/UBSan and CI pass;
- Slice 1 learning PDF and tag created.

Section 8 - Slice 2 - L3 book reconstruction

Build correctness first, then optimize.

1. Implement a simple ownership-safe reference book.
2. Define and test add/modify/remove semantics from observed Bitstamp events.
3. Add debug-only invariants: no duplicate IDs, correct side/price membership, FIFO links, size sums, uncrossed best prices when expected and lookup/list agreement.
4. Replay only snapshot-backed continuous segments.
5. Aggregate reconstructed L3 into levels and compare with Bitstamp same-venue L2/checkpoints.
6. Only after correctness, benchmark candidate level storage and introduce the pool/intrusive layout if measurements justify it.

Do not use Coinbase as the equality oracle. Cross-venue comparison is only a separate sanity chart.

Section 9 - Slice 3 - deterministic replay core

Begin single-threaded. Determinism is more valuable than concurrency during correctness work.

- `Feed`, `Clock`, `ExecutionVenue` and `Strategy` are injected boundaries.
- Replay event ordering is explicit; no system clock leaks into replay.
- Fees and inventory accounting are present from the first fill model.
- No-op strategy conserves event count, position and PnL exactly.
- A small hand-calculated strategy scenario matches expected orders and fills.

Add the SPSC queue after the deterministic baseline passes. Measure throughput and p50/p99/p99.9 latency before and after; explain memory ordering and false sharing rather than copying a known implementation.

Section 10 - Slice 4 - queue and execution model

L3 queue treatment

For visible existing orders, L3 order IDs and queue order can identify whether a later cancellation was ahead of or behind an observed/hypothetical insertion point. Therefore the old universal "cancel location is unknowable" premise is no longer correct for the primary L3 path.

Remaining uncertainty includes hidden liquidity, simultaneous/tied events, capture gaps, venue priority rules, replace semantics and counterfactual market impact. ADR 0008 must be rewritten after trade/order joining demonstrates which Bitstamp events are fills, partial fills, cancels and replacements.

For an L2-only fallback, retain optimistic/pessimistic/proportional cancellation assumptions and report their spread.

Latency treatment

Separate:

- local decoder/book/strategy processing latency measured with a monotonic clock;
- observed network/ack round trips only if an execution adapter provides them;
- scenario latency used for sensitivity when no trustworthy live measurement exists.

Do not claim a fitted end-to-end latency distribution from venue timestamps alone.

Section 11 - Slice 5 - corpus validation

Dataset construction

- Use capture sessions, not individual events, as the independence unit.
- Split chronologically by session/day; never randomly mix adjacent events across train/test.
- Fit/tune on earlier sessions and report once on held-out sessions.
- Censor at segment ends, gaps and incomplete observation horizons.
- Record every exclusion rule before seeing headline metrics.

Metrics

- Brier score and reliability curve for fill-within-horizon probability;
- log loss as a secondary proper score;
- calibration error with bin counts shown;
- time-to-fill distribution/survival analysis for censored outcomes;
- bootstrap confidence intervals resampled by session or time block, not individual correlated

events;

- signed post-fill adverse selection and opportunity cost for unfilled orders;
- stratification by spread, volatility, side, queue depth and time of day.

"Predicted fill price versus actual fill price" is not the primary metric for passive limit orders because their limit price is largely known. Arrival-price slippage, adverse selection and opportunity cost are more meaningful.

Section 12 - Slice 6 - operational path

Core operational deliverable:

- Boost.Beast Bitstamp live market-data feed;
- same C++ strategy/replay core with a local paper execution venue;
- risk checks and an explicit order state machine;
- a versioned stream of book, trade, order, model and health telemetry;
- a small control surface for replay and local paper-order commands;
- CLI diagnostics and saved analysis files that work without a GUI.

The telemetry contract must define message versions, timestamps, source/mode, snapshot versus incremental updates, ordering, backpressure and disconnect behavior. A slow or crashed dashboard must not block or corrupt the engine. Keep commands separate from observation messages so that a chart subscriber cannot accidentally become an execution authority.

Optional external execution requires a separately verified test environment and credentials. It must not block corpus validation or the local paper venue. ZeroMQ remains an optional transport; choose it only if its process separation and publish/subscribe model fit the measured dashboard requirements.

Section 13 - Slice 7 - interactive dashboard

Build a local browser-based dashboard after the replay, model and live-data contracts are stable. The preferred split is a C++ engine process plus a Python presentation layer using plotting and web-dashboard libraries. Matplotlib remains useful for reproducible saved reports; an interactive plotting layer may be used for live charts and replay controls.

Required modes

1. **Historical replay:** choose a validated capture, play/pause, seek where the file format allows it, change replay speed and inspect model predictions against later observed outcomes.
2. **Live observation:** display the current Bitstamp book, trades, spread, feed health and model forecasts without sending exchange orders.
3. **Local paper execution:** submit and cancel hypothetical orders against the live or replayed book and inspect their state, queue estimate and simulated fills.

Required views

- best bid/ask, spread, mid-price and recent trades;
- aggregated depth and selected L3 price-level queue;
- hypothetical order state and estimated queue position;
- fill-probability forecast, later outcome and calibration summaries;
- time-to-fill, adverse selection and opportunity-cost plots;
- replay timestamp/speed plus message-rate, latency, gap and reconnect health indicators.

Architectural rules

- Python must not reconstruct the authoritative book or independently calculate fills.
- Historical and live sources feed the same normalized C++ engine interfaces.
- The engine runs headless for CI, benchmarks and long captures.
- UI commands are validated by an application/controller boundary before reaching the engine.
- The first dashboard is local-only; public hosting, accounts and remote control are out of scope.

Exit gate

- one historical session can be replayed and visually inspected from start to finish;
- live read-only data updates the same views without changing model code;
- local paper-order submit/cancel/fill state is visible and tested;
- saved plots reproduce headline held-out results;
- disconnecting or slowing the dashboard does not alter deterministic replay results or stop the engine;
- the dashboard remains responsive at the measured target update rate through aggregation or bounded sampling rather than dropping engine correctness events.

Section 14 - Data collection plan

The 30-second clean capture is sufficient for current decoder/record development. Validation requires broader sessions.

Target, subject to storage and API stability:

- at least 12 one-hour BTC/USD sessions;
- weekday and weekend coverage;
- Asia, Europe and US trading-hour coverage;
- calm and volatile regimes identified after capture without discarding inconvenient sessions;
- manifest, hashes and validator result for every session;
- at least one second instrument only after `InstrumentSpec` is tested.

Capture collection may run unattended while reading, documentation or non-overlapping learning work proceeds. It must not distract from the current C++ implementation task. Keep raw corpora out of Git and back them up outside the workspace.

Section 15 - CI and quality gates

Activate now:

- a narrowly scoped no-floating-price guard over core price/event/PnL interfaces;
- header compile contracts for trivial copyability and enum representation;
- clean-build unit tests under ASan/UBSan on Linux;
- capture validator tests using tiny synthetic good/gap/error manifests.

Activate later:

- clock-use guard when `Clock` exists;
- allocation counter for measured hot paths;
- deterministic replay hash across repeated runs;
- benchmark regression thresholds only after stable hardware/environment methodology exists.

A text grep for `double|float` is a temporary guard and must avoid comments, test demonstrations, and legitimate research code. The lasting protection is domain types plus API/decoder tests.

Section 16 - Learning contract

The project exists to teach C++, so assistance follows this sequence for learning-critical code:

1. State the problem, invariants and relevant language concepts in English.
2. Fuaad sketches the data layout or algorithm and writes the first attempt.
3. Compile/run tests and inspect the first meaningful failure.
4. Review correctness, ownership, complexity and undefined behavior.
5. Provide a full implementation only when explicitly requested after an attempt.
6. At each slice end, re-derive the core logic without the editor and complete a short quiz.

Mechanical CMake, CI, documentation and capture bookkeeping may be implemented on explicit request, with an explanation of what changed.

Section 17 - Immediate next actions

1. Finish the red-green `EventKind` and normalized `OrderEvent` exercise without copying a body.
2. Put permanent structural `static_assert` contracts beside the completed types; keep runtime behavior in GoogleTest.
3. Add the temporary scoped CI price-type guard after the event header compiles.
4. Design `InstrumentSpec` and the exact decimal parser through tests.
5. Specify and obtain a short joined Bitstamp order+trade capture before designing fill labels.
6. Amend ADR 0008 only after the joined sample is analyzed.
7. Keep the clean 1,433-event capture as the positive fixture source and the legacy chain break as the negative fixture source.

Section 18 - Sources and evidence

- Original `Trading_Engine_Coding_Plan (1).pdf`, August 2026.
- `Quant_Developer_Roadmap_Fuaad.pdf`, current five-month C++/quant-development roadmap.
- Bitstamp API documentation: `https://www.bitstamp.net/api/`.
- Bitstamp WebSocket documentation: `https://www.bitstamp.net/websocket/v2/`.
- Coinbase Exchange authentication and channel documentation:

`https://docs.cdp.coinbase.com/exchange/websocket-feed/authentication` and `https://docs.cdp.coinbase.com/exchange/websocket-feed/channels`.

- Repository ADRs 0001-0010, capture manifests and validator output as of 2026-08-09.